ParkSure: An Online Vehicle Parking Reservation System with a Hold-Back Buffer and Trust Score

PALLE YASHASWINI
Roll No. 23K91A05H9
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
[EMAIL]

MOHAMMED MUNEEB AHMED
Roll No. 24K95A0518
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
[EMAIL]

KAMASANI MANISHA
Roll No. 24K95A0513
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
[EMAIL]

POTHAGANI NITHYYA BHARGAVA
Roll No. 23K91A05J6
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
[EMAIL]

V PALLAVI
[DESIGNATION]
(project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
[EMAIL]

***Abstract*---**Drivers in dense cities lose time looking for a free bay, and a reservation only helps if the reserved bay is really free on arrival. This paper presents ParkSure, a web platform in which drivers search availability for a time window, reserve a bay in advance, pay online and check in and out, while operators manage locations and see analytics. Its availability engine admits a request only if overlapping bookings plus a hold-back buffer stay below capacity, moves a driver to the nearest alternate location by haversine distance when a lot is full on arrival, and adjusts fees and access through a 0-5 Trust Score. The system has 22 API routes and five tables and is checked by 70 automated tests. A discrete-event simulator, verified decision by decision against the real engine, compares five buffer policies. In it the shipped dynamic buffer refuses the most requests, arrivals with no bay stay rare under every policy, and the Trust lockout is the largest source of refusals. Our own probes also found an over-booking race and an unauthenticated payment-confirmation route.

***Index Terms*---**buffer policy, haversine distance, parking reservation, reputation system, smart parking, web application.

I. INTRODUCTION

Searching for a free space adds to urban congestion and emissions, and efficient parking management is a recurring theme of smart-city research [1], [2]. In many car parks today the operator keeps manual registers or a cash counter, has no live view of occupancy and no record of demand, and the driver learns whether a space exists only on arrival. Advance reservation turns that search into a guaranteed bay, but it creates a failure of its own: a reserved bay may still be held by a previous driver who overstays, the "No Park" situation described by Cabri et al. [2]. The base paper of this project optimises how vehicles are assigned to spaces inside one large managed facility [1]; the work that proposes a reservation buffer [2] is evaluated only in simulation. What is missing is an operating reservation platform for ordinary urban parking that joins search, booking, payment and operator analytics with an overstay-aware availability rule, and that reports how it actually behaves.

This paper makes four contributions. (1) ParkSure, a tested full-stack platform (FastAPI, React, SQLite; 22 routes, five tables) for search, reservation, payment, check-in, check-out and administration. (2) A Smart Availability Engine that implements the reservation buffer and star-reputation ideas of Cabri et al. [2] in a product, and adds a haversine nearest-alternate move with automatic credit and a trust-based fee multiplier. (3) Verification: 70 pytest tests, a simulator that agrees with the engine on 10,300 decisions, and measured response times. (4) A comparison of five buffer policies and a plain list of the defects that our own probes found. Section II reviews related work, Section III the method, Section IV the results and Section V concludes.

A. System Hardware

All measurements were taken on one machine: 12th-generation Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro, no GPU, other programs running.

B. Software Requirements

Python 3.14.7, FastAPI 0.141.1 [3], SQLAlchemy 2.0.52, SQLite, APScheduler 3.11.3, python-jose 3.5.0 for JSON Web Tokens [4], bcrypt 5.0.0 [5], the Stripe SDK 15.6.1 [6] in test mode, React 19 with Vite 8 and Node 24.19.0 for the console, pytest and Playwright with Chrome for tests and screenshots.

II. LITERATURE SURVEY

Base paper. Jemmali et al. [1] address parking in medical cities, where each vehicle carries a different number of people. They assign vehicles to parking spaces so that the total number of people per space, not the number of vehicles, is balanced, and minimise the total excess of the loads over the smallest load. Visitors register a request by web or mobile application with their passenger count, receive a QR code, and at the gate sensors compare the declared count with the occupancy seen by cameras before a scheduler allocates a space. A four-stage grouping procedure and six grouping-permutation heuristics are evaluated on nine instance classes and 2,790 test cases: the best algorithm has the highest best-solution share (68.2 %), the variant GVb matches or beats the baseline MR on 1,547 instances (55.4 %), and execution times are below a millisecond. In our reading the paper solves allocation inside one facility and does not treat payment, overstays or operator analytics.

Technical basis. Cabri et al. [2] study reservation-based parking in which only vehicles with a reservation may enter. A Reservation Module keeps a buffer of non-reservable bays per area; when a driver finds the area full, the system suggests exactly one nearby area and books it automatically, and if that fails the driver cannot park ("No Park"). The dynamic rule raises the buffer by one whenever a vehicle leaves after its reservation ended, up to a maximum, with a periodic reset. A Reward System keeps a 0-5 star rating: B benefits (on-time departures) add a star, W warnings (overstays) remove one, the rating changes the fee, and a low rating with a recent violation suspends booking. In a SUMO simulation [7] of 80 spots in 8 areas, with 80 to 110 vehicles, 25 to 50 % of them overstaying by one hour, and five runs, any buffer beats none and the dynamic buffer gives the best trade-off between failed parks and rejected reservations; the results are given as figures, not tables.

Related work. Reservation has been approached as scheduling, optimisation, pricing, auction and service design. Doulamis et al. [8] model requests as time intervals that a scheduler accepts or rejects, a view that matches the classical interval-scheduling problem [9]; Mejri et al. [10] formulate slot assignment as a multi-objective integer programme over walking distance, vehicle proximity and congestion; iParker [11] adds flexible reservation times with dynamic pricing; Somani et al. [12] ask for part of the fee up front to prevent false bookings; and Safitri et al. [13] build a mobile reservation system with a heuristic and the haversine formula. Auction-based systems let agents or drivers bid for spaces [14], [15], [16]; SmartParking [17] offers on-the-fly reservation with real-time availability; Chen et al. [18] analyse the efficiency and equity of reservation; and surveys organise the field [19]. Table I summarises what each work reports, using only what the two papers above state about them.

TABLE I
RELATED WORK COMPARED

Work	Approach	Data	Reported result	Limitation
Jemmali et al. [1]	Passenger-count zone balancing, 6 heuristics	9 classes, 2,790 test cases	Best 68.2 %; GVb ties or beats MR on 55.4 %	One facility; no payment, overstay
Cabri et al. [2]	Buffer of non-reservable bays + star reward	SUMO, 80 spots, 8 areas, 5 runs	Any buffer beats none; dynamic best (figures)	Simulation only; preliminary
Doulamis et al. [8]	Interval scheduling of requests	Not stated in [2]	Not stated in [2]	Overstays not described
Mejri et al. [10]	Multi-objective integer programme	Not stated in [2]	Not stated in [2]	No operating platform described
Kotb et al. [11]	Flexible times, dynamic pricing	Not stated in [2]	Not stated in [2]	Punctuality-based fees not described
Somani et al. [12]	Part-payment to confirm a booking	Not stated in [2]	Not stated in [2]	Overstays not addressed
Safitri et al. [13]	Mobile app, heuristic, haversine	Not stated in [2]	Not stated in [2]	No buffer or reputation
Rizvi, Kokolaki [14], [15], [16]	Auction-based allocation	Not stated in [2]	ASAP: better utilisation (per [2])	Not a fixed-price service

Positioning. Three observations follow from Table I. First, the two papers that matter most for ParkSure answer different questions: the registered base paper [1] decides where each vehicle goes inside a facility, while Cabri et al. [2] decide how many bays a booking system should hold back. ParkSure takes the booking-system side, so its rules come from [2] and its architecture from the web-booking interface that [1] also assumes. Second, none of the other works in the table is described, in the two papers that cite them, as handling the overstay that makes a reserved bay unavailable; the deposit of Somani et al. [12] deters false bookings but not late departures. Third, the cited results are mostly not stated, so the comparison in Section IV is made against the two base papers only and does not pretend to rank ParkSure among the others.

Gap. The surveyed work either optimises allocation inside one managed facility, proposes a buffer and reputation mechanism that is evaluated only in simulation, or describes reservation components one at a time. A general platform that combines search, booking, payment, an overstay-aware availability rule and operator analytics, and that measures its own behaviour, is not reported. ParkSure implements that platform and tests it.

III. METHODOLOGY

A. Architecture and Implementation

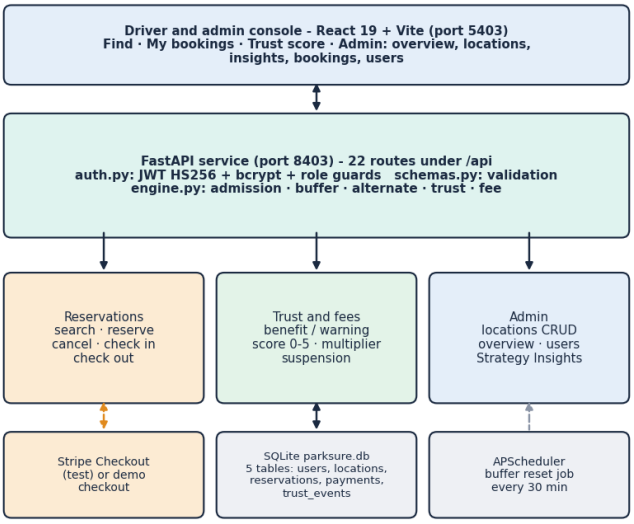


Fig. 1. ParkSure architecture: React console, FastAPI service with 22 routes and the availability engine, SQLite with five tables, a scheduler job and Stripe or the demo checkout.

The console (Vite proxy from port 5403 to the API on 8403) has landing, login, find parking, my bookings and trust-score pages for drivers and five admin pages. The API follows the usual REST style: JSON bodies validated by pydantic (422 on bad input), bearer tokens with role guards (401, 403), 409 when a window is full. Passwords are hashed with bcrypt at cost 12. SQLAlchemy maps users, parking_locations, reservations, payments and trust_events to one SQLite file. When no Stripe key is configured an offline demo checkout keeps every flow usable, and the interface says so. A payment record is created with every reservation (paid at once when account credit covers the fee, pending otherwise) and moves to refunded when a paid booking is cancelled, the fee then returning as account credit. For operators, the Strategy Insights route replays the stored booking requests of each location in time order under four buffer settings (none, 1, 3 and the dynamic rule) and counts

requests declined by admission and requests admitted but unable to park, so the same input always gives the same output.

Four implementation decisions are worth stating with their reasons. SQLite keeps installation to one file and is enough for a single-process demonstration, but it limits concurrent writers; the admission rule is a plain count over an indexed interval query because it is easy to audit and costs 0.33 ms for 100 stored bookings (Section IV); the nearest-alternate search issues one availability query per other location, which is simple and correct but grows linearly with the number of locations; and the engine is kept in one module with no web imports so that the simulator and the tests can call the same functions.

B. Admission, Buffer and Alternate

A reservation is an interval [s, e) at a location l with capacity C_l and buffer b_l. With n_ov(s, e) the number of reserved or active bookings that overlap the interval, the request is admitted only if

$$n_{ov}(s, e) + b_l < C_l, \qquad (1)$$

otherwise it is stored as declined with the reason. The buffer starts at 1; every on-time check-out (at most ten minutes after e) raises it by one up to 5, and a scheduler job running every 30 minutes resets it to 1 once 24 hours have passed. Note that this grows the buffer on punctual departures, whereas the paper [2] grows it on late ones; we keep the shipped rule and evaluate both. At check-in, if the number of parked vehicles already equals C_l, the engine searches the other locations that pass the admission test for the same window and picks the nearest by the haversine distance [20]

$$a = 2R \operatorname{asin}\left(\sqrt{\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cos\varphi_2 \sin^2(\Delta\lambda/2)}\right), \quad R = 6371\ \text{km}. \qquad (2)$$

The driver is booked there as an active alternate; if no location qualifies, the booking is marked failed and the fee is credited to the account.

C. Trust Score and Fee

Each driver starts with a score T = 3 on a 0-5 scale. An on-time check-out records a benefit and three consecutive benefits add one point; a late check-out resets the streak and records a warning, and two warnings remove one point. The fee is

$$fee = \max(0.25, (e - s) / 1\ \text{h}) \times price_l \times m(\text{round } T), \quad m = \{1.75, 1.5, 1.25, 1.0, 0.95, 0.9\} \text{ for } T = 0...5. \qquad (3)$$

A driver with T < 1.5 and a warning in the last 14 days is refused and paused for 24 hours at each attempt. Account credit offsets the fee before payment.

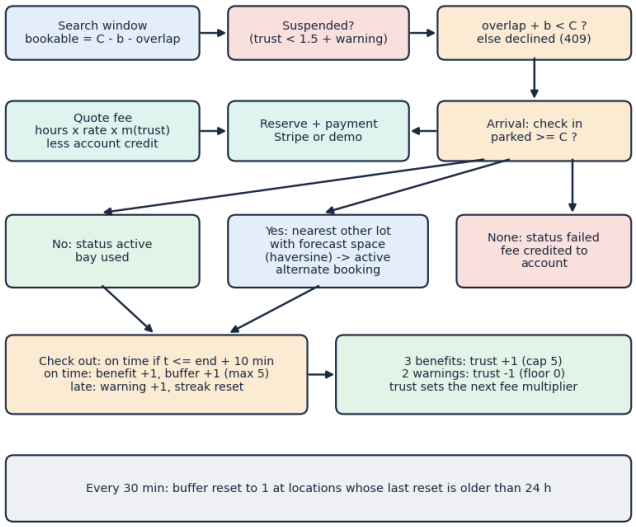


Fig. 2. Main flow of ParkSure: availability search, suspension and admission checks, fee quote and payment, arrival (direct, alternate or failed), check-out and the trust and buffer updates.

D. Data and Evaluation Protocol

ParkSure uses no external dataset. Its demo database, produced by the product's own seed script, holds 5 locations, 9 users and 347 reservations over 60 days, and is therefore synthetic. We evaluate in four ways. (i) 70 pytest tests (29 engine, 36 API, 5 simulator) and 27 smoke checks. (ii) A discrete-event simulator re-creates the setting of [2]: 8 areas of 10 bays, 80 to 110 vehicles making 10 parks of 1 to 3 hours, bad vehicles leaving one hour late, two long stops outside the system, and runs the ParkSure rules including the Trust Score. It is not SUMO, and what the paper does not give (arrival process, lead time 15 to 60 minutes, gaps, uniform area choice) is our assumption, fixed before the runs. Each cell is the mean of 30 runs. Its admission, trust and suspension functions agree with the real engine on 10,300 decisions with 0 mismatches. The simulator processes three event types in time order: a request (suspension check, then admission), an arrival (direct parking, alternate move or failure) and a departure (buffer and trust update, then the vehicle's next request). Every request ends in exactly one outcome, refused by admission, refused by the lockout, parked directly, parked at an alternate, or no park, and a test checks that the five counts sum to the number of requests. Utilisation is occupied bay-hours divided by the 80 bays times the length of the run. (iii) Response times on 127.0.0.1 with 200 sequential requests per endpoint. (iv) Probes for concurrency and security on a throw-away database.

IV. RESULTS

A. Buffer Policies in the Simulator

Five policies are compared: static buffers of 0, 1 and 3 bays, the shipped dynamic rule (buffer grows on on-time departures) and the paper's rule (grows on late departures). "Refused" counts requests rejected by admission, "no bay" counts arrivals that found no bay (no alternate, or an alternate that was itself full), and "success" is the share of all requests

that ended with a parked car. Table II gives two operating points.

TABLE II
SIMULATOR RESULTS (MEAN OF 30 RUNS PER CELL)

Policy	Refused, 100 veh., 25 % bad	Success %	Refused, 50 % bad	No bay, 50 % bad	Success %
Static 0	30	82.7	19	3.33	69.0
Static 1	53	80.8	34	1.30	68.0
Static 3	158	72.5	102	0.17	64.3
Dynamic (ParkSure)	277	63.0	162	1.43	60.3
Dynamic (paper rule)	165	71.5	136	1.10	62.2

At 100 vehicles and 25 % overstaying drivers the shipped dynamic rule refuses 277 of 1,000 requests, the paper's rule 165, static 3 158 and no buffer 30; it raises the buffer after every punctual departure, up to the cap of five. Arrivals with no bay are rare for every policy (at most 3.3 per run of about 936 requests at 50 % overstay): the alternate move absorbs the overflow. The cost shows as the alternate itself being full: static 0 produces 2.9 such events per run against 0.0 for static 3, because the shipped code books the alternate without re-checking its bays. Utilisation falls from 33.5 % (static 0) to 26.8 % (shipped dynamic) at 100 vehicles. The Trust lockout matters more than the buffer: at 50 % overstay it refuses 211 to 271 requests per run, and switching it off lifts the success rate of static 0 from 69 % to 98 %.

Demand pressure acts as expected. From 80 to 110 vehicles the refusals of static 0 rise from 8 to 49 per run and those of the shipped dynamic rule from 108 to 383, while the success rate of static 0 falls from 84 % to 81 % and that of the shipped rule from 74 % to 57 %. With more overstaying drivers the refusals by admission of the shipped rule fall from 214 to 162, but only because the refusals by the lockout rise from 97 to 211: the requests are refused earlier, not served. Cabri et al. also see fewer rejected reservations as the share of bad vehicles grows and attribute it to lower turnover [2]; in our runs the cause is the lockout, and we make no claim about theirs.

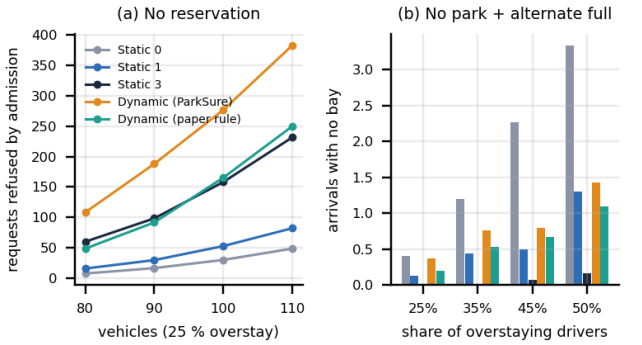


Fig. 3. Buffer policies in the simulator: (a) requests refused by admission against the number of vehicles (25 % overstaying); (b) arrivals with no bay against the share of overstaying drivers. Legend as in Table II.

Replaying the 60-day seeded history under four buffers gives, as unable-to-park / declined, 8/16 (no buffer), 4/24 (fixed 1), 0/40 (fixed 3) and 0/60 (dynamic), the same ordering on synthetic data. For the Trust Score, a driver who ends 30 % of sessions late reaches a mean score of 1.8 after 100 daily sessions, a mean fee multiplier of 1.36, and 80 % are suspended at some point; at 50 % the mean score is 0.1 and all are suspended.

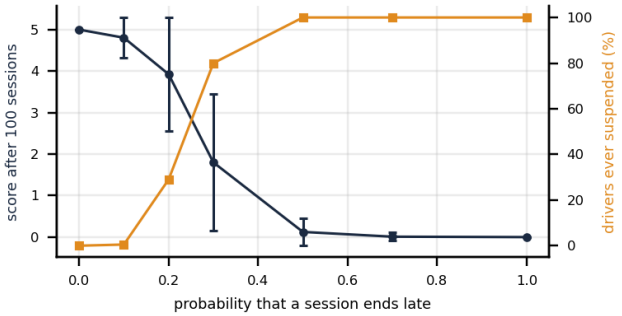


Fig. 4. Trust Score dynamics of one driver over 100 daily sessions (200 simulated drivers per point): mean final score and share of drivers ever suspended against the probability of a late session.

The curve shows how sharp the rule is: at a late-session probability of 0.1 the mean score stays at 4.8 and 0.5 % of drivers are ever suspended, at 0.2 the mean is 3.9 and 29 % are suspended, and from 0.3 upwards most drivers are locked out within about 26 sessions. Because the lockout repeats at every attempt while the score is below 1.5 and the last warning is under 14 days old, a driver who is suspended cannot earn the benefits that would restore the score before the 14 days pass.

B. Response Time and Tests

TABLE III
TESTS AND MEASURED RESPONSE TIME

Measure	Result
Automated tests	70 of 70 pass; 27 of 27 smoke checks
Simulator against engine	10,300 decisions, 0 mismatches
Search availability	5.7 ms mean, 6.6 ms p95
Reserve	19.6 ms mean, 28.2 ms p95
Check in / check out	14.9 / 14.6 ms mean
Login (bcrypt, cost 12)	220 ms mean
Own bookings list (about 400 rows)	180 ms mean
Admin all bookings (300 rows)	273 ms mean
Admin Strategy Insights	60 ms mean
Admission decision, in process	0.33 ms mean
Nearest alternate, 49 locations	18.6 ms mean
Search throughput, 8 threads	131 requests/s

The 29 engine tests cover the haversine distance, every fee multiplier, the admission rule with and without buffers, the nearest-alternate choice, the trust updates and caps, suspension and the buffer reset; the 36 API tests cover registration, login, role guards, location management, search, reservation, cancellation with credit, check-in with the alternate and the failed outcome, check-out, suspension, payment, administration and validation errors. The tests assert the intended behaviour of the code as shipped; they do not cover concurrency, which is why the race below was found only by a separate probe.

Ordinary calls take under 20 ms. The two slow list endpoints are slow because every row issues two extra database queries, a defect rather than a necessity. The admission decision itself is sub-millisecond, which echoes the sub-millisecond execution that the base paper reports for its heuristics [1], but the two solve different problems and the numbers are not comparable.

C. Output Screens

The screens were captured from the running application on a fresh database with real data produced by the flows themselves. In Fig. 5 a second driver holds a booking at Kiosk Bay, a one-bay lot already occupied by an earlier driver, so on check-in the engine books Central Station Garage, 0.1 km away, marks the first booking unable to park and links the new booking to it. Fig. 6 is the operator view of the stored-history replay quoted above.

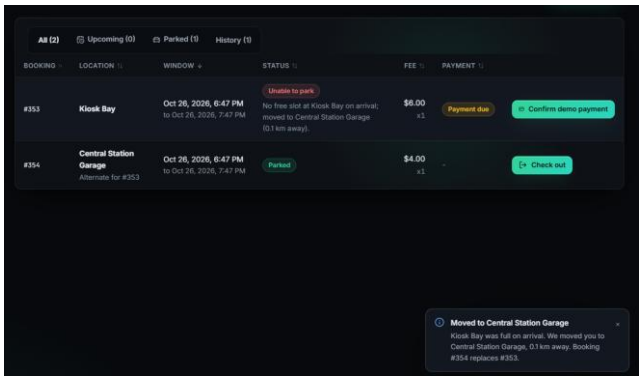


Fig. 5. Real screen: a driver arrives at the occupied one-bay Kiosk Bay, is moved to Central Station Garage 0.1 km away, and the original booking is marked unable to park.

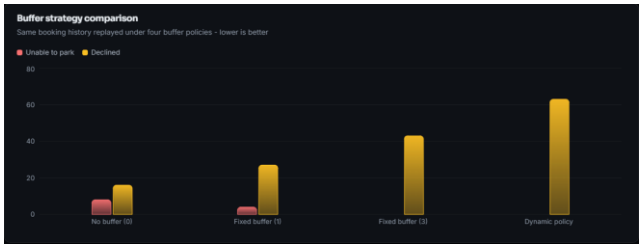


Fig. 6. Real screen: Strategy Insights, the stored booking history replayed under four buffer policies (unable to park and declined).

D. Comparison with the Base Papers

The registered base paper [1] optimises zone load balance on 2,790 test instances; ParkSure does not implement its grouping algorithms, so no score is comparable and we do not claim to match or beat it. Both are web-booking systems with a central allocation step and fast decisions. Against Cabri et al. [2], whose results are figures, the direction "any buffer helps" is partly visible here (no bay events at 50 % overstay: 3.3 for no buffer against 0.2 for static 3), but "dynamic is best" is not reproduced for the shipped rule, which refuses the most. The settings differ in every respect: our simulator is not SUMO, the arrival process is assumed, the Trust Score is included, the alternate move is part of the system, and we use 30 runs where the paper uses five.

E. Failure Cases and Observed Limits

TABLE IV
DEFECTS FOUND BY THE TEAM'S OWN PROBES

Probe	Observed
20 simultaneous requests, 1 bookable bay	1 of 10 trials accepted 2 bookings
Payment confirm route without login	Booking marked paid: yes
Token forged with the default JWT secret	Admin route accepted: yes
30 wrong passwords in a row	All refused, no lockout or rate limit
Booking window in the past	Accepted: yes
Check-in 25 days before the window	Accepted: yes
Injection-style inputs, other user's booking	Rejected (422), blocked (404)

Our probes found real defects. In 1 of 10 trials of 20 simultaneous requests for one bookable bay, two bookings were accepted, because the admission count and the insert are not atomic. The demo payment route can be called without a login and marks any booking paid; a token forged with the default secret is accepted; failed logins are not rate limited; bookings that start in the past and check-in 25 days early are accepted. The suspension lasts at least 14 days after the last warning, not one session. There is no vehicle profile and no downloadable receipt, so two of the six project objectives are only partly met. The data are synthetic, availability is a forecast without sensors, and there was no user study, no hardware gate and no SUMO run.

V. CONCLUSION AND FUTURE SCOPE

ParkSure delivers a working path from availability search to reservation, payment, check-in, alternate re-allocation, check-out and operator analytics, verified by 70 tests and by a simulator that agrees with the engine. Against the objectives, registration, search, booking, cancellation, payment in test mode, the admin dashboard and the evaluation are achieved; vehicle profiles and receipts are not. The main finding is that the shipped dynamic buffer, which grows on punctual departures, refuses the most requests and gives no clear gain over static buffers, and that the Trust lockout dominates refusals once a quarter of drivers overstay. Limits are the synthetic data, the simulator in place of SUMO and the defects listed above.

For an operator the practical reading is cautious. The product is usable for a demonstration and for studying reservation policy, but before any real deployment the payment route must be closed, admission made atomic, and the buffer rule chosen deliberately: a static buffer of one or three bays refused far fewer requests than the shipped dynamic rule in our simulator, at the price of more alternate moves.

Future work: make admission atomic and close the payment route; add vehicle profiles and PDF receipts; test the paper's rule in the product and re-check the alternate's bays; shorten the lockout to one session; predict overstays from history; and measure against real parking data and a SUMO scenario.

REFERENCES

[1] M. Jemmali, L. K. B. Melhim, A. Ammar, T. R. Gadekallu and W. Boulila, "Smart parking for medical cities via passenger-aware allocation and fast grouping-permutation heuristics," IEEE Open Journal of the Computer Society, vol. 7, pp. 1106-1118, 2026, doi: 10.1109/OJCS.2026.3693801.

[2] G. Cabri, M. Montangero, F. Muzzini and R. Wang, "Reservation based smart parking management," arXiv:2605.17650 [cs.MA], May 2026.

[3] S. Ramirez, "FastAPI documentation," [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 2026).

[4] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[5] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conf., FREENIX Track, 1999, pp. 81-91.

[6] Stripe, Inc., "Stripe Checkout and webhooks documentation," [Online]. Available: https://docs.stripe.com (accessed Oct. 2026).

[7] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flotterod, R. Hilbrich, L. Lucken, J. Rummel, P. Wagner and E. Wiessner, "Microscopic traffic simulation using SUMO," in Proc. 21st IEEE Int. Conf. on Intelligent Transportation Systems (ITSC), 2018, pp. 2575-2582.

[8] N. Doulamis, E. Protopapadakis and L. Lambrinos, "Improving service quality for parking lot users using intelligent parking reservation policies," in Proc. 27th Int. Conf. on Advanced Information Networking and Applications Workshops, IEEE, 2013, pp. 1392-1397.

[9] J. Kleinberg and E. Tardos, Algorithm Design. Boston, MA, USA: Addison-Wesley, 2006.

[10] N. Mejri, M. Ayari, R. Langar and L. Saidane, "Reservation-based multi-objective smart parking approach for smart cities," in Proc. 2016 IEEE Int. Smart Cities Conf. (ISC2), 2016, pp. 1-6.

[11] A. O. Kotb, Y.-C. Shen, X. Zhu and Y. Huang, "iParker - a new smart car-parking system based on dynamic resource allocation and pricing," IEEE Transactions on Intelligent Transportation Systems, vol. 17, no. 9, pp. 2637-2647, 2016.

[12] A. Somani, S. Periwal, K. Patel and P. Gaikwad, "Cross platform smart reservation based parking system," in Proc. 2018 Int. Conf. on Smart City and Emerging Technology (ICSCET), IEEE, 2018, pp. 1-5.

[13] R. R. Safitri, A. Pratiarso and A. Zainudin, "Mobile-based smart parking reservation system with rate display occupancy using heuristic algorithm and haversine formula," in Proc. 2020 Int. Electronics Symp. (IES), IEEE, 2020, pp. 332-339.

[14] S. R. Rizvi, S. Zehra, R. Mukkamala and S. Olariu, "ASAP: An agent-assisted smart auction-based parking system in Internet of Things," in Proc. 9th ACM Symp. on Design and Analysis of Intelligent Vehicular Networks and Applications, 2019, pp. 1-8.

[15] S. R. Rizvi, S. Zehra and S. Olariu, "MaPark: A multi-agent auction-based parking system in Internet of Things," IEEE Intelligent Transportation Systems Magazine, vol. 13, no. 4, pp. 104-115, 2020.

[16] E. Kokolaki, M. Karaliopoulos and I. Stavrakakis, "Trading public parking space," in Proc. IEEE Int. Symp. on a World of Wireless, Mobile and Multimedia Networks, 2014, pp. 1-6.

[17] G. Yan, W. Yang, D. B. Rawat and S. Olariu, "SmartParking: A secure and intelligent parking system," IEEE Intelligent Transportation Systems Magazine, vol. 3, no. 1, pp. 18-30, 2011.

[18] R. Chen, G. Gao, L.-J. Kang and L.-Y. Zhang, "Efficiency and equity analysis on parking reservation," Transportation Research Part E: Logistics and Transportation Review, vol. 187, art. no. 103606, 2024.

[19] M. Aljohani, S. Olariu, A. Alali and S. Jain, "A survey of parking solutions for smart cities," IEEE Transactions on Intelligent Transportation Systems, vol. 23, no. 8, pp. 10012-10029, 2021.

[20] R. W. Sinnott, "Virtues of the haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.